

Daniel Parker

April 21, 2025

CS 470 Project Two Conference Presentation: Cloud Development

<https://www.youtube.com/watch?v=vaHZ5MECJS0>

My name is Daniel Parker, and I'm here to walk you through the journey of migrating a traditional full stack application to a cloud-native, serverless solution using AWS. This talk is for developers, start-up teams, and anyone curious about scaling applications efficiently.

Containerization

Our full stack application was first containerized and tested locally using Docker Compose before migrating each component to AWS.

As you can see, our Docker dashboard displays each component of our project organized into its respective container.

Docker allowed us to encapsulate both the frontend (web) and backend (api) services, which really streamlined the development process by allowing us to run the entire stack with a single command.

Overall, this really helped make the project consistent across development, staging, and production and ultimately made it easier to detect issues early.

Orchestration

Here, you will see our web hosting on Docker Compose above and AWS S3 below.

Migrating to a serverless architecture allowed us to eliminate traditional infrastructure management altogether.

For frontend deployment, we used AWS S3 to host static files. This choice came with several advantages:

- Automatic scalability to handle fluctuations in user traffic
- Built-in redundancy for high availability and fault tolerance
- Seamless integration with services like CloudFront, API Gateway, and Lambda

This orchestration removed the need to manage web servers and allowed us to focus on code and user experience.

The Serverless Cloud

"Serverless" refers to a development model where the cloud provider handles all server operations behind the scenes—this includes provisioning, scaling, and maintenance.

The biggest advantages of going serverless include automatic scaling, reduced operational costs, and a faster development process.

You only pay for what you use, making it ideal for startups.

To serve our frontend, we used AWS S3. Unlike local storage—which is limited by physical hardware—S3 offers high availability, durability, and global access.

It's also fully integrated with other AWS services like Lambda, which we'll cover shortly.

As part of our cloud migration, we transitioned from a traditional MongoDB document-based model to DynamoDB, which uses a key-value and document hybrid model.

This required a shift in moving from flexible JSON documents to a more structured, partition-key-based schema.

We set up separate DynamoDB tables for users, questions, and answers, each designed with performance and scalability in mind.

For example, the Answer table uses a unique id as the partition key, and includes attributes for questionId, vote counts, and the actual answer text.

To interact with these tables, we created Lambda functions like UpsertAnswer, which used the AWS SDK to perform PUT and UPDATE operations based on the HTTP method.

These functions were tested independently using the Lambda console before being wired up to API Gateway endpoints.

Once integration was complete, we enabled CORS and fine-tuned method responses to ensure the frontend could communicate securely and effectively with our backend.

In our original app, MongoDB used a flexible JSON schema with rich relationship definitions and built-in data typing.

The Answer.ts model defined an interface and a detailed model structure using LoopBack conventions.

In our serverless migration, we replaced this with DynamoDB. The UpsertAnswer Lambda function builds a key-value style parameter map using UpdateCommand.

There is no enforced schema in DynamoDB – we manage structure and defaults in code.

MongoDB gave us more structure. DynamoDB gave us more scalability and flexibility for a serverless architecture.

On the left, you will see our DynamoDB script on AWS, and on the right is our local MongoDB script.

One of the biggest advantages of moving to AWS Lambda and DynamoDB is elasticity—our app now scales automatically, so we never overpay for unused capacity.

Since AWS uses a pay-per-use billing model, we only incur costs when our app is actively used, which is a game-changer for cost efficiency.

To protect our cloud-native application from unauthorized access, we implemented several key security measures at different layers of the architecture:

We began by establishing IAM roles and policies.

Each Lambda function was assigned an execution role, and that role had attached policies defining exactly which AWS services it could interact with.

For example, our UpsertAnswer function can only update specific items in the Answer table, and nothing else.

We also created custom policies to tightly control access to DynamoDB, CloudWatch, and other services.

These policies are written in JSON and follow the principle of least privilege, ensuring that each service only does what it absolutely needs to do.

To secure the communication between services:

- The connection between API Gateway and Lambda is protected using IAM, and CORS is enabled to safely allow requests from our Angular frontend.
- The link between Lambda and DynamoDB is controlled by tightly scoped permissions in IAM policies.
- Our S3 bucket is secured using bucket policies that restrict public access and only allow specific roles to upload or retrieve files if needed.

Thanks for listening! I'd love to connect and hear how others are building cloud-native apps. Let's keep pushing the limits of what's possible with serverless.